

CAS4133 LLM · 기말 대비 · Chapter 12

Chapter 12. Reinforcement Learning with LLMs

RL 기초 개념 → Policy Gradient 유도 → Reward-to-go·Baseline·Advantage → PPO(-Clip) → TRL 구현 → GRPO → 2026 Post-training(GLM-5, On-policy Distillation)

 강의일: 2026-06-01(도입) · 2026-06-08(본강의)

 예상 비중: 높음 (수식 최다 챕터)

 분량: 본문 8섹션 + 문제 17

목차

1. RL의 핵심 개념 — Agent, Environment, State, Action, Reward
2. RL 문제의 수학적 정식화 — Trajectory, Return, $J(\theta)$
3. 기본 Policy Gradient의 유도 — Log-derivative Trick
4. Vanilla Policy Gradient — Reward-to-go, Baseline, Advantage
5. PPO — Clipped Surrogate Objective
6. PPO in RLHF — TRL 구현 (Value Head, torch.no_grad)
7. GRPO — Group Relative Policy Optimization
8. 2026년 LLM Post-training — GLM-5, On-policy Distillation
9. 총정리 비교표: REINFORCE vs PPO vs GRPO
10. 🎯 작년 기말 출제: Training 수식 작성 (Problem 3, 15점)
11. 예상문제 (T/F 9 + Pick-all 8) + 작년 기출 T/F 3

RL은 이제 LLM **post-training**의 표준 패러다임이다. 대표 사례: **RLHF** (Ouyang et al., NeurIPS 2022 — OpenAI, 알고리즘은 **PPO**)와 **DeepSeek-R1** (DeepSeek-AI, arXiv 2025.01 — 알고리즘은 **GRPO**). 지금까지 수업에서는 목적함수(objective function)를 어떻게 만드는지만 다뤘고, 이번 장에서는 구체적인 RL 알고리즘이 LLM과 어떻게 작동하는지를 다룬다.

교수 강조 (강의 발언)

"이 수업은 RL 수업이 아니므로 LLM 맥락 이해에 필요한 **최소한의 기본 개념만** 간략히 다룬다. RL을 더 깊이 배우고 싶으면 이(Lee) 교수님의 강화학습 수업을 들어라. 이번 강의는 RL 분류 체계(model-free vs model-based 등) 전체가 아니라 **policy optimization**에만 초점을 맞춘다 — LLM을 RL로 학습시키는 가장 직관적인 방법이기 때문."

두 등장인물: Agent & Environment

정의

Environment: agent가 살아가며 상호작용하는 세계(world that agent lives in and interacts with). **Agent**는 매 step마다 환경의 **state**를 관찰(observe)하고 **action**을 결정한다. 환경은 agent의 행동에 의해 변하며(*may also change on its own* — 스스로 변할 수도 있음), agent는 환경으로부터 **reward signal**을 받는다.

- 매 step의 상호작용: 상태 s_t 에서 agent가 행동 a_t 를 취함 $\rightarrow$ 상태가 s_{t+1} 로 전이 $\rightarrow$ 환경이 보상 r_t 를 줌.
- Agent의 목표 = **누적 보상(cumulative reward)의 최대화**. 이 누적 보상을 **return**이라 부른다.
- **RL의 목표**: agent가 목표를 달성하는 행동을 학습할 수 있는 방법(알고리즘)을 찾는 것.
- 예: 게임 = environment, 플레이어/캐릭터 = agent, 버튼 누르기 = action, 화면/캐릭터 위치 변화 = state 변화, 레벨 클리어 = goal.

Policy (정책)

정의

Policy: agent가 어떤 행동을 취할지 결정하는 데 쓰는 **규칙 또는 함수**(rule or function used by agent to decide what actions to take). **deterministic**(결정론적: $a_t = \mu(s_t)$, 예: rule-based)일 수도, **stochastic**(확률적: $a_t \sim \pi(\cdot | s_t)$, 행동을 샘플링)일 수도 있다.

- Deep RL에서는 보통 **parameterized policy** π_θ 를 사용 — 즉 DNN으로 정책을 학습.
- **LLM과의 대응 (시험 단골 매핑):**
 1. **Policy = LLM**
 2. **State = input tokens** (초기 상태 = input prompt; 출력 토큰이 프롬프트에 concatenate되며 상태가 갱신됨)
 3. **Action = generation of next token** (다음 토큰 분포에서 디코딩 알고리즘으로 행동 생성)

Trajectory (궤적) & State Transition

정의

Trajectory τ : 상태와 행동의 연속(sequence of states and actions), $\tau = (s_0, a_0, s_1, a_1, \dots)$. 상태 전이 규칙 $(s_t, a_t) \rightarrow s_{t+1}$ 이 존재한다고 가정한다 (예: 게임에서 왼쪽 버튼 → 화면 속 캐릭터 이동).

Reward & Return

정의

Reward: agent의 바람직한 행동을 촉진하는 신호(signal to promote desired behavior). 수학적으로 $r_t = R(s_t, a_t, s_{t+1})$ — 상태 s_t 에서 행동 a_t 로 s_{t+1} 이 됐을 때 그 변화가 목표 달성에 얼마나 유익한지.

Return (누적 보상) — finite-horizon vs infinite-horizon

$$R(\tau) = \sum_{t=0}^T r_t \quad (\text{finite-horizon undiscounted return})$$

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t, \quad \gamma \in (0, 1) \quad (\text{infinite-horizon discounted return})$$

γ 는 **discount factor**. 무한 horizon에서 γ 를 도입하지 않으면 합이 **무한대로 발산**하므로, 스칼라 값으로 만들기 위해 반드시 필요하다.

Value Function & Q Function (조건부 보상 함수)

Conditional reward functions (정책 π 하에서)

$$V^\pi(s) = \mathbb{E}_{\tau \sim \pi}[R(\tau) \mid s_0 = s] \quad (\text{Value function: 특정 state에 조건부})$$

$$Q^\pi(s, a) = \mathbb{E}_{\tau \sim \pi}[R(\tau) \mid s_0 = s, a_0 = a] \quad (\text{Q function: state \& action에 조건부})$$

$$V^\pi(s) = \mathbb{E}_{a \sim \pi}[Q^\pi(s, a)] \quad (\text{Natural relationship})$$

Value function = 상태 s 에서 시작해 정책 π 대로 행동했을 때의 **기대 return**. Q function은 거기에 첫 행동 a 까지 고정한 것. 정책 하에서 가능한 행동들에 대해 Q의 기댓값을 취하면 V가 된다.

★ 시험 핵심

① return = **cumulative** reward (단일 step reward가 아님). ② LLM 매핑: policy=LLM / state=input tokens / action=next-token generation. ③ V는 state에만, Q는 state+action에 조건부. ④ $V^\pi(s) = \mathbb{E}_{a \sim \pi}[Q^\pi(s, a)]$. ⑤ discount factor는 infinite-horizon에서 발산을 막기 위한 것.

⚠ 함정 포인트 (T/F 대비)

- "The goal of an RL agent is to maximize the immediate reward at each step." → **False**. The goal is to maximize the *cumulative* reward over a trajectory (the return), not the per-step reward.
- "In the LLM-as-RL framing, the state corresponds to the LLM's parameters." → **False**. The state is the *input tokens* (prompt + generated tokens so far); the policy is the LLM itself, and the action is generating the next token.
- "A policy must be stochastic." → **False**. A policy can be deterministic (e.g., rule-based) or stochastic; deep RL typically uses a parameterized (often stochastic) policy π_θ .
- "The discount factor γ is required in the finite-horizon return." → **False**. It is introduced for the *infinite*-horizon case to keep the return finite.

- "The Q function is conditioned only on the initial state." → **False**. That is the value function; Q is conditioned on both the initial state *and* the initial action.

Trajectory의 확률

Probability of trajectory (Markov assumption 하)

$$P(\tau \mid \theta) = \rho_0(s_0) \prod_{t=0}^T P(s_{t+1} \mid s_t, a_t) \pi_\theta(a_t \mid s_t)$$

$\rho_0(s_0)$ = 초기 상태 분포, $P(s_{t+1} \mid s_t, a_t)$ = 환경의 상태 전이 확률, $\pi_\theta(a_t \mid s_t)$ = 정책의 행동 확률. **Markov assumption**: 현재 상태는 바로 직전 상태(와 행동)에만 조건부.

Expected Return과 최적화 문제

Expected return & RL 최적화 문제

$$J(\pi_\theta) = \int_{\tau} P(\tau \mid \theta) R(\tau) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

$$\pi^* = \arg \max_{\pi} J(\pi) \quad (\text{Optimal policy})$$

기댓값은 정책 π_θ 로 샘플링된 궤적에 대해 취한다 — 즉 J 는 그 정책 자체에 대한 평가다. 누적 보상이 높으면 그 정책이 목표 달성에 좋은 정책.

Policy Gradient에 의한 최적화

Gradient ascent (정책 경사 상승)

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J(\pi_{\theta})|_{\theta_k}$$

$\nabla_{\theta} J(\pi_{\theta})$ 가 **policy gradient**. 손실 최소화(gradients descent)와 달리 return을 **최대화**하므로 gradient **ascent**(+부호). policy gradient로 정책을 최적화하는 알고리즘 = **policy gradient algorithms** (예: **PPO**, **GRPO**).

그러나 policy gradient 계산은 간단하지 않다 — J 자체가 기댓값이므로 **기댓값의 gradient**를 구해야 하기 때문. 따라서 두 단계가 필요하다:

- 1 **해석적 gradient 유도** (deriving analytical gradient of policy performance) — 기댓값 형태의 $J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$ 의 gradient를 닫힌 식으로 유도.
- 2 **표본 추정** (forming sample estimate from finite # of interactions) — 복잡한 분포에선 기댓값을 수학적으로 계산할 수 없으므로, 유한 개의 상호작용으로 **empirical sample mean**으로 추정.

★ 시험 핵심

① 궤적 확률 = 초기분포 $\times \prod$ (전이확률 $\times$ 정책확률). ② $J(\pi_\theta)$ 는 **그 정책으로 샘플링한 궤적**에 대한 기대 return. ③ 정책 갱신은 gradient **ascent**. ④ PPO·GRPO는 둘 다 policy gradient 계열 알고리즘.

⚠ 함정 포인트 (T/F 대비)

- "Policy gradient methods update the policy via gradient descent on the expected return." → **False**. They use gradient *ascent* because the goal is to *maximize* expected return (equivalently, descent on the negative return — but the slide formulation is ascent).
- "The trajectory probability depends only on the policy π_θ ." → **False**. It also depends on the initial state distribution ρ_0 and the environment transition probabilities $P(s_{t+1} \mid s_t, a_t)$.
- "GRPO is not a policy gradient algorithm because it has no value model." → **False**. Both PPO and GRPO are policy-gradient(policy optimization) algorithms; the difference is only in how the advantage is estimated.

📖 교수 강조 (강의 발언)

"이 부분(유도 과정 자체)을 요구하는 시험 문제는 내지 않겠다 — 흥미와 이해를 위한 것이다." → 유도 과정의 재현은 출제 안 되지만, 최종 결과식과 각 트릭의 이름·역할(log-derivative trick, 환경 함수의 gradient=0, Monte Carlo 추정)은 T/F로 충분히 나올 수 있다.

유도에 필요한 세 가지 사실

① Log-probability of trajectory (곱 → 합)

꺾적의 로그 확률

$$\log P(\tau \mid \theta) = \log \rho_0(s_0) + \sum_{t=0}^T \left(\log P(s_{t+1} \mid s_t, a_t) + \log \pi_\theta(a_t \mid s_t) \right)$$

② Log-derivative trick

Log-derivative trick

$$\nabla_\theta P(\tau \mid \theta) = P(\tau \mid \theta) \nabla_\theta \log P(\tau \mid \theta)$$

$\nabla \log f = \nabla f / f$ 에서 바로 나온다. 확률의 gradient를 "확률 × 로그확률의 gradient"로 바꿔, 기댓값 안으로 gradient를 넣을 수 있게 해주는 핵심 트릭.

③ 환경 함수의 gradient는 0

Gradients of environment functions are zero

$$\nabla_\theta \log P(\tau \mid \theta) = \underbrace{\nabla_\theta \log \rho_0(s_0)}_{=0} + \sum_{t=0}^T \left(\underbrace{\nabla_\theta \log P(s_{t+1} \mid s_t, a_t)}_{=0} + \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right)$$

매개변수 θ 는 **정책에만** 들어있다. ρ_0 와 상태 전이 확률은 환경의 함수로 θ 를 포함하지 않으므로 θ 에 대한 gradient가 0 → 궤적 로그확률의 gradient는 **정책 항들의 합으로 환원**된다.

기본 Policy Gradient (결과식 — 암기 필수)

Basic Policy Gradient

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

직관: 어떤 행동이 **높은 보상**을 받았다면 그 행동의 로그확률 gradient에 큰 가중치가 곱해져 그 행동이 더 장려되고, 낮은 보상이면 덜 장려된다.

Monte Carlo 추정 (sample estimate)

Estimated policy gradient (표본 평균)

$$\hat{g} = \frac{1}{|\mathcal{D}|} \sum_{\tau \in \mathcal{D}} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau), \quad \mathcal{D} = \{\tau_i\}, \tau_i \sim \pi_{\theta}$$

기댓값을 닫힌 형식으로 계산할 수 없으므로 **Monte Carlo estimation**: 현재 정책 π_{θ} 로 유한 개의 궤적을 샘플링해 표본 평균으로 대체. 궤적이 **현재 정책에서** 샘플링된다는 점이 on-policy의 핵심.

★ 시험 핵심

① 결과식 $\nabla J = \mathbb{E}[\sum_t \nabla \log \pi_{\theta}(a_t | s_t) R(\tau)]$. ② log-derivative trick의 역할. ③ 환경 항(ρ_0 , transition)은 θ 를 포함하지 않아 gradient 0. ④ 기댓값 → 샘플 평균(Monte Carlo)으로 추정.

⚠ 함정 포인트 (T/F 대비)

- "Computing the basic policy gradient requires differentiating the environment's transition probabilities." → **False**. The gradients of environment functions (ρ_0 , transition dynamics) w.r.t. θ are zero, since θ parameterizes only the policy.
- "The expectation in the policy gradient can be computed in closed form." → **False**. It is approximated by a sample mean over trajectories sampled from the current policy (Monte Carlo estimation).

- "In the basic policy gradient, trajectories may be sampled from any policy." → **False**. The derivation assumes $\tau \sim \pi_\theta$ — the *current* policy (on-policy). Reusing samples from an old policy requires an importance-sampling ratio (as in PPO).

Vanilla Policy Gradient — Reward-to-go, Baseline, Advantage

① Reward-to-go: 시간 단계 문제 해결

기본 유도는 $R(\tau)$ (궤적 전체의 누적 보상)를 모든 시간 단계 t 의 행동에 곱한다. 그러나 시점 t 의 행동은 **그 이전의 보상**에 영향을 줄 수 없다 → 행동 이전의 보상까지 반영하는 것은 부정확. 따라서 행동 **이후**의 보상만 합산:

Reward-to-go policy gradient (more accurate)

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{t'=t}^T R(s_{t'}, a_{t'}, s_{t'+1}) \right]$$

$\sum_{t'=t}^T r_{t'} = \text{reward-to-go}$: 시점 t 이후부터의 누적 보상만 고려. t 이전의 보상은 반영하지 않는다.

② Baseline: 기댓값 불변 + 분산 감소

Baseline의 수학적 근거 — 임의의 $b(s_t)$ 에 대해

$$\begin{aligned} \mathbb{E}_{a_t \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b(s_t)] &= 0 \\ \Rightarrow \nabla_{\theta} J(\pi_{\theta}) &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(\sum_{t'=t}^T r_{t'} - b(s_t) \right) \right] \end{aligned}$$

state만의 함수 $b(s_t)$ (= **baseline**)는 기댓값이 0이므로 더하거나 빼도 **policy gradient의 기댓값은 불변**. 그러나 좋은 baseline은 추정의 **분산(variance)**을 감소시킨다 (empirically 확인됨). 가장 일반적인 선택 = **on-policy value function** $V^{\pi}(s_t)$ — 누적 보상의 절대값이 아니라 평균 대비 **상대적 차이**에 집중하게 해 준다.

📖 교수 강조 (강의 발언)

"이 등식($\mathbb{E}[\nabla \log \pi_{\theta} b(s_t)] = 0$)의 유도 역시 시험에서 요구하지 않는다. 중요한 건 **baseline을 넣어**도 기댓값은 그대로이고, 분산만 줄어든다는 사실이다."

③ Q function 치환과 Advantage

Q function 형태 → Advantage 형태

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q^{\pi}(s_t, a_t) \right]$$

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi}(s_t, a_t) \right], \quad A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$$

① 기대 return $R(\tau)$ 를 **Q function**으로 대체해도 동일한 **policy gradient**가 됨이 증명 가능. ② 여기에 value function을 baseline으로 빼면 **advantage function** $A = Q - V$ 형태가 된다.

정의

Advantage function $A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$: 행동 a_t 가 그 상태에서의 **평균적 행동보다 얼마나 더 나은지**(how much better it is than average)를 포착. 평균보다 좋으면 $A > 0$, 나쁘면 $A < 0$. 누적 보상과 달리 미래 상태 전체를 직접 계산할 필요 없이 현재 시간 단계 안에서 행동의 품질을 추정할 수 있다.

④ Φ_t 의 선택에 따른 알고리즘 이름

policy gradient를 $\nabla_{\theta} J = \mathbb{E}[\sum_t \nabla \log \pi_{\theta}(a_t | s_t) \Phi_t]$ 로 쓸 때, **Φ_t 의 선택**에 따라 알고리즘 이름이 달라진다:

Φ_t 선택	알고리즘 이름
$R(\tau) - (\text{할인 없는}) \text{ 전체 누적 보상}$	REINFORCE
$\sum_{t'=t}^T r_{t'} - b(s_t) - \text{baseline 삽입}$	REINFORCE with baseline
$A^{\pi}(s_t, a_t) = Q - V - \text{advantage 사용}$	Advantage Actor-Critic 계열

⑤ Value function의 근사와 VPG 알고리즘

value function의 기댓값도 닫힌 형식으로 계산 불가 → **별도의 신경망(value model $\hat{V}_{\phi}$)**으로 근사하여 학습한다. Vanilla Policy Gradient(VPG) 알고리즘의 전체 흐름:

- 1 **궤적 샘플링** — 현재 정책 π_{θ_k} 로 trajectory 집합 $\mathcal{D}_k$ 수집
- 2 **Reward-to-go 계산** — 보상 함수로 $\hat{R}_t$ 계산
- 3 **Advantage 계산** — 현재 value 모델 $\hat{V}_{\phi_k}$ 기반으로 $\hat{A}_t$ 추정
- 4 **Policy gradient 계산 & 정책 갱신** — 누적 보상 대신 advantage 사용, gradient ascent로 θ 갱신
- 5 **Value model 갱신** — 계산된 reward-to-go에 대한 회귀(MSE)로 ϕ 갱신

📖 교수 강조 (강의 발언)

"정책에만 관심이 있어도 **별도의 value model을 추가로 학습**해야 한다. 이것이 **PPO의 주요 병목(bottleneck)**이 된다 — PPO도 value function을 필요로 하는 알고리즘이기 때문이다." → GRPO의 동기로 이어지는 복선.

★ 시험 핵심

① reward-to-go = 시점 t **이후** 보상만 (이전 보상 제외). ② baseline은 기댓값 **불변** + 분산 **감소** (state만의 함수여야 함). ③ 가장 흔한 baseline = on-policy value function. ④ $A = Q - V$, 평균 행동 대비 우수성. ⑤ Φ_t 선택 → REINFORCE / REINFORCE w. baseline / Advantage Actor-Critic. ⑥ value function은 별도 신경망으로 근사.

⚠ 함정 포인트 (T/F 대비)

- "Adding a baseline $b(s_t)$ changes the expected value of the policy gradient." → **False**. $\mathbb{E}[\nabla \log \pi_{\theta} b(s_t)] = 0$, so the expectation is unchanged; only the *variance* of the estimate is reduced.
- "A baseline can be an arbitrary function of both state and action." → **False**. The zero-expectation identity holds for functions of the *state* $b(s_t)$ only; an action-dependent baseline would bias the gradient.
- "Reward-to-go includes rewards obtained before time step t ." → **False**. It sums rewards from t onward only — actions cannot affect rewards already received.
- "The advantage $A^{\pi} = V^{\pi} - Q^{\pi}$." → **False**. $A^{\pi} = Q^{\pi} - V^{\pi}$ (방향 주의!). Positive advantage = better than the average action at that state.
- "Using the advantage instead of $R(\tau)$ yields a biased estimate of the policy gradient." → **False**. Replacing $R(\tau)$ with Q^{π} and subtracting V^{π} as a baseline can

be proven to give exactly the same expected policy gradient.

(Proximal Policy Optimization — 슬라이드 표기: "Proximal Point Optimization")

⚠ 표기 주의

- 강의 슬라이드와 교수 발언 모두 PPO를 "**Proximal Point Optimization**"으로 표기/발음했다. 일반적으로(원논문 Schulman et al., OpenAI) PPO는 "**Proximal Policy Optimization**"이다. 시험에서 어느 쪽 표기가 나와도 같은 알고리즘으로 인식할 것. 둘 다 정답으로 처리될 가능성이 높지만, 본 수업 기준 표기는 슬라이드를 따른다.

동기 (Motivation)

RL 학습의 최대 난제 중 하나는 **불안정성(instability)**. PPO의 질문: "*현재 정책에서 너무 멀리 벗어나 붕괴(collapse)하지 않으면서, 가능한 가장 큰 개선 step을 뺄 수 있을까?*" (take biggest possible improvement step without stepping too far from current policy)

- Note #1:** 같은 질문에 먼저 답한 선행 연구 = **TRPO** (Trust Region Policy Optimization, Schulman et al., ICML 2015). 그러나 너무 복잡하고 계산 비용이 3~4배 이상 비싸 적용이 어려움.
- Note #2:** PPO 개발자 = **John Schulman** (OpenAI). 두 변형 존재: **PPO-Penalty**(이전 정책으로 부터의 급격한 갱신을 막는 KL penalty 추가, TRPO에 가까움) & **PPO-Clip**(배포가 더 쉬움). 수업은 **PPO-Clip**에 집중.

📖 교수 강조 (강의 발언)

"직관: gradient가 100으로 계산되더라도 10만큼의 갱신만 허용한다 — **단일 step 안에서의 갱신량을 제한**함으로써 갑작스러운 갱신과 그로 인한 문제를 방지한다."

PPO-Clip Objective

PPO-Clip 정책 갱신

$$\theta_{k+1} = \arg \max_{\theta} \mathbb{E}_{s,a \sim \pi_{\theta_k}} [L(s, a, \theta_k, \theta)]$$

$$L(s, a, \theta_k, \theta) = \min \left(\underbrace{\frac{\pi_{\theta}(a | s)}{\pi_{\theta_k}(a | s)}}_{r_t(\theta): \text{ratio}} A^{\pi_{\theta_k}}(s, a), \text{clip} \left(\frac{\pi_{\theta}(a | s)}{\pi_{\theta_k}(a | s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta_k}}(s, a) \right)$$

$r_t(\theta) = \pi_{\theta}(a|s) / \pi_{\theta_k}(a|s) =$ **이전 정책 대비 현재 정책의 확률 비율** (importance sampling ratio). $\epsilon = \text{clip}$ 폭 하이퍼파라미터. 어드밴티지는 **이전 정책 π_{θ_k} 기준**으로 계산.

단순화된 동치 표현 (g 함수)

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s, a), g(\epsilon, A^{\pi_{\theta_k}}(s, a)) \right), \quad g(\epsilon, A) = \begin{cases} (1 + \epsilon) A \\ (1 - \epsilon) A \end{cases}$$

경우 분석 — clip이 작동하는 방식 (출제 1순위)

경우	Objective	정책이 하려는 것	clip의 역할
Advantage > 0 (평균보다 좋은 행동)	$\min \left(\frac{\pi_{\theta}}{\pi_{\theta_k}}, 1 + \epsilon \right) A$	$\pi_{\theta}(a s)$ 를 증가 시키려 함 (좋은 행동을 더 장려)	min 이 objective가 증가할 수 있는 양에 상한 을 둬 $\rightarrow$ ratio를 $1 + \epsilon$ 이상 키워도 이득 없음 $\rightarrow$ 새 정책이 이전 정책에서 멀리가도 이득이 없다
Advantage < 0 (평균보다 나쁜 행동)	$\max \left(\frac{\pi_{\theta}}{\pi_{\theta_k}}, 1 - \epsilon \right) A$	$\pi_{\theta}(a s)$ 를 감소 시키려 함 (나쁜 행동 억제)	max 가 objective가 감소할 수 있는 양에 하한 을 둬 $\rightarrow$ ratio가 $1 - \epsilon$ 보다 작아져도 이득 없음 $\rightarrow$ 역시 이전 정책에서 멀어질 유인이 없다

두 경우 모두 결론은 동일: **새 정책 θ 는 이전 정책 θ_k 에서 멀리 가는 것으로부터 이득을 얻지 못한다** (new policy does not benefit by going far away from old policy).

PPO 알고리즘 & 한계

- Pseudo-code는 VPG와 구조가 거의 동일 — 궤적 샘플링, reward-to-go, advantage 계산은 같고 **유일한 차이는 PPO-Clip objective**로 정책을 갱신하는 부분.
- 여전히 advantage가 필요하므로 **별도의 value model 학습 필요** (value model 갱신 단계 존재).

- 주의: clip을 써도 새 정책이 이전 정책에서 너무 멀어지는 것이 여전히 가능(it is still possible to end up with new policy too far from old policy) → 실무에서는 추가 트릭(additional tricks)이 널리 채택됨.

LLM에서의 관례: Clip + KL penalty 병용

📖 교수 강조 (강의 발언)

"PPO-Clip은 collapse 방지를 위해 설계됐지만, LLM에서는 망각(forgetting)을 막기에 충분하지 않은 것으로 알려져 있다. 따라서 LLM 맥락에서는 PPO-Clip을 쓰더라도 reference policy로부터의 KL penalty도 함께 고려한다 — 즉 PPO-Clip 항과 PPO-Penalty 항을 함께 쓰는 것이 LLM에서의 관례다." (GRPO objective의 KL 정규화 항이 바로 그 형태)

★ 시험 핵심

① TRPO(복잡·고비용) → PPO(간단화, John Schulman, OpenAI). ② 변형 2개: PPO-Penalty(KL), PPO-Clip(수업 초점). ③ clipped objective: $\min(rA, \text{clip}(r, 1 - \epsilon, 1 + \epsilon)A)$. ④ $A > 0$ 이면 min이 상한, $A < 0$ 이면 max가 하한 — 양쪽 모두 "멀리 갈 유인 제거". ⑤ PPO는 value model 필요. ⑥ clip만으로 너무 먼 정책 방지가 보장되지 않음 → 추가 트릭 / LLM에선 KL penalty 병용.

⚠ 함정 포인트 (T/F 대비)

- "PPO's clipping guarantees that the new policy never moves far from the old policy." → **False**. The slides explicitly note it is *still possible* to end up with a policy too far from the old one; additional tricks are adopted in practice.
- "When the advantage is positive, the clip term puts a lower bound on how much the objective can increase." → **False**. The *min* puts an *upper limit* (ceiling) on the increase. The lower-bound (max) case applies when the advantage is *negative*.
- "PPO removes the need for a value function." → **False**. PPO still computes advantages with a learned value model — that is its main bottleneck and the motivation for GRPO.
- "TRPO was abandoned because it was unstable." → **False**. TRPO answers the same stability question; its problem is *complexity and computational cost* (3–4× more expensive), which PPO simplifies.

- "In the PPO ratio $\pi_{\theta}/\pi_{\theta_k}$, the denominator is the reference (SFT) policy." → **False**. The denominator is the *old policy* π_{θ_k} from the previous iteration. The reference policy appears in the (separate) KL-penalty term used in RLHF/GRPO.
- "PPO-Clip alone is sufficient to prevent forgetting in LLM training." → **False**. For LLMs, a KL penalty w.r.t. the reference policy is additionally used because clipping alone is not enough.

6 PPO in RLHF — TRL 구현 (Implementation in Practice)

TRL (Transformer Reinforcement Learning)

- **TRL**: **Hugging Face**가 직접 공개한, transformers 라이브러리 위의 **full-stack LLM 학습 라이브러리**. DPO·PPO 등 검증된 학습 알고리즘들이 구현되어 있음.
- 사용법: **config와 trainer 함수만 바꾸면** 알고리즘 교체 가능 (예: `DPOConfig` + `DPOTrainer`).
- 단, 계산 면에서 최적은 아닐 수 있음 — 최근 연구자들은 **veRL**을 많이 사용 (TRL보다 훨씬 빠른 것으로 알려짐). 교육·학습 목적으로는 TRL이 잘 구성되어 있음.

교수 강조 (강의 발언)

"LLM에게 PPO 코드를 짜게 하면 되지 않냐고 물을 수 있다. 하지만 최신 LLM조차 **세상에 없는 연구 코드 (out-of-domain)**는 종종 잘못 구현한다. 동료 교수의 학생도 코드 오류 때문에 아이디어가 실패한 줄 알았다가, 코드를 고치자 잘 작동했다. **어떤 코드든 이해하고 비판·검토할 수 있어야 한다** — LLM에게 골격을 짜게 하더라도 검토는 너희가 해야 한다." (이 TRL 코드 부분 자체는 "시험에 나오지 않는다, 학습과 이해를 위한 것"이라고 명시함)

TRL PPOTrainer의 train 함수 (lines 346–684) — 단계별 구조

- 1 **Step 0 (413–432). Rollout** — 주어진 query마다 현재 정책으로 **여러 응답(multiple responses) 생성**(local rollout). `torch.no_grad` 하에서 수행 → gradient를 만들지 않음, 순수 샘플링.
- 2 **Step 1 (433–460).** 응답 후처리 + **target policy와 reference policy의 logits** 계산.
- 3 **Step 2 (460–490).** **rewards**(line 469, reward model로) & **values**(line 465, value model로) 계산.
- 4 **Step 3 (491–505).** 손실 계산을 위한 후처리.
- 5 **Step 4 & 5 (506–520).** **RLHF objective 계산** — log-ratio(비율)를 계산하고 **KL 정규화를 보상에 더함** (Ouyang et al., NeurIPS 2022의 RLHF 목적과 정확히 일치).
- 6 **Step 6 (521–535).** 계산된 value들로 **advantages & returns** 계산 ($A^\pi = Q^\pi - V^\pi$).
- 7 **Step 7 (537–562).** 갱신을 위한 target policy의 forward pass.

8 Step 8 (563–576). **value head**를 갱신하는 손실 계산 (LM head와 별도).

9 Step 9 (577–606). PPO objective에 따라 **policy model**을 갱신하는 손실 계산.

핵심 트릭: 별도 value model 대신 Value Head

정의

LM head: hidden state(로짓)를 **vocabulary에 대한 확률**로 변환하는 부분. **Value head**: hidden state를 **value(스칼라 가치)**로 변환하는 부분. LLM RLHF에서는 별도의 value model을 두는 대신 **LLM 위에 value head만 추가**한다.

왜? RLHF-PPO를 그대로 하려면 ① policy model ② reference model(정규화용) ③ value model ④ reward model — **최대 4개의 모델**을 적재해야 한다. 별도 value model까지 두면 메모리·계산 부담이 너무 크므로, value head 추가로 근사한다. (단, 그만큼 value model의 capacity가 전체 모델 대비 크게 줄어드는 한계 → GRPO 동기 중 하나)

★ 시험 핵심

① TRL = Hugging Face의 RL 학습 라이브러리, trainer/config 교체만으로 알고리즘 변경. ② veRL = TRL보다 빠른 대안. ③ rollout(샘플링)은 `torch.no_grad` 하에서 — gradient 생성 없음. ④ RLHF-PPO는 policy + reference + value + reward 모델이 관여. ⑤ 별도 value model 대신 **value head**를 LLM 위에 추가 (LM head는 vocab 확률, value head는 가치).

⚠ 함정 포인트 (T/F 대비)

- "In TRL's PPO implementation, the rollout (response generation) step also backpropagates gradients through the policy." → **False**. Rollout is performed under `torch.no_grad` — it is pure sampling and produces no gradients.
- "In LLM RLHF, the value function is typically implemented as a fully separate model of the same size as the policy." → **False**. To avoid loading yet another full model, a small *value head* is added on top of the LLM (alongside the LM head).
- "The LM head converts hidden states into scalar values for advantage estimation." → **False**. The *LM head* maps to a probability distribution over the vocabulary; the *value head* maps to values.

- "TRL is developed by OpenAI." → **False**. TRL is released by Hugging Face. (PPO 알고리즘 자체가 OpenAI/John Schulman.)

동기: value estimation 제거

GRPO는 DeepSeek이 제안한 RL 알고리즘 (**DeepSeekMath**, Shao et al., arXiv 2024.04에서 제안 → **DeepSeek-R1**, Guo et al., arXiv 2025.01에서 핵심 알고리즘으로 사용). PPO의 동기(급격한 갱신 방지)와 달리, GRPO의 동기는 **value estimation에 대한 의존성 제거**다.

왜 value estimation이 문제인가? (두 가지 이유 — 출제 포인트)

1. **상당한 메모리·계산 부담** (substantial memory and computational burden) — LLM RL은 이미 여러 모델(policy/reference/reward)을 적재해야 하는데 value model까지 추가됨. value head로 근사하면 부담은 줄지만 **capacity가 크게 줄어드는** 문제.
2. **reward는 보통 마지막 토큰에서 계산되지만, value는 각 토큰마다 계산되어야 함** (while reward is usually calculated using last token, value is calculated on each token) → 정확한 value 추정이 복잡해짐. 자연어에서는 단일 토큰 생성이 지금은 좋아 보여도 몇 단계 뒤 나쁜 것으로(혹은 그 반대로) 판명될 수 있어 토큰 단위 value 추정이 매우 비직관적이다.

핵심 아이디어: 그룹 평균을 baseline으로

정의

GRPO는 PPO의 value function 추정을 제거하고, 대신 **여러 샘플 출력(roll-out)의 평균 보상을 baseline으로 사용**한다. 논리: advantage의 본질은 **현재 행동(생성)의 상대적 품질 추정**이므로, 평균과 비교할 다른 방법(=그룹 평균)이 있으면 value function의 역할을 대체할 수 있다. "Group Relative"라는 이름은 **그룹 내부에서의 상대 비교**로 advantage를 추정하기 때문.

- 단일 궤적 대신 **한 query당 여러 출력을 생성** → 평균 보상 계산 → 평균보다 좋은 출력엔 보너스, 나쁜 출력엔 페널티.

GRPO Objective (수식)

GRPO objective — query q 마다 old policy $\pi_{\theta_{\text{old}}}$ 에서 그룹 $\{o_1, \dots, o_G\}$ 샘플링

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}_{q, \{o_i\} \sim \pi_{\theta_{\text{old}}}} \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}, 1 + \beta, 1 - \beta \right) \right] \right\}$$

왼쪽 min/clip 부분은 **PPO와 동일한 구조**. 차이는 ① advantage $\hat{A}_{i,t}$ 를 value model 없이 **그룹 내 상대 보상**으로 계산, ② **reference policy에 대한 KL 정규화 항**(계수 β)이 objective에 직접 포함된다는 점.

Group-relative advantage (그룹 평균·표준편차 정규화)

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\{r_1, \dots, r_G\})}{\text{std}(\{r_1, \dots, r_G\})}$$

각 출력 o_i 의 보상 r_i 에서 그룹 평균을 빼고 그룹 표준편차로 나눠 정규화. **그룹 내부에서만**(inside each group only) 상대적으로 계산된다.

GRPO의 KL divergence 추정 (다른 estimator 사용)

$$\mathbb{D}_{KL}[\pi_{\theta} \parallel \pi_{\text{ref}}] = \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})} - \log \frac{\pi_{\text{ref}}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})} - 1$$

바닐라 KL 대신 위의 **unbiased estimator**를 사용 (교수: "그다지 중요하지 않다"고 언급 — 다른 추정 방식을 쓴다는 사실만 알면 됨).

GRPO 알고리즘 흐름

- 1 이전 iteration의 매개변수를 $\pi_{\theta_{\text{old}}}$ 로 초기화
- 2 $\pi_{\theta_{\text{old}}}$ 가 query당 G 개의 출력을 샘플링
- 3 각 출력에 대해 **reward model**로 보상 계산
- 4 보상들로 **group-relative advantage** $\hat{A}_{i,t}$ 계산 (value model 불필요!)
- 5 GRPO objective(PPO와 유사 + KL 정규화)를 최대화하여 정책 갱신

장단점과 변형

장점

단점

value model 학습·추론 제거 → 메모리·계산 절감, **구현이 훨씬 쉽고 PPO보다 안정적**으로 알려짐

단일 query 갱신을 위해 **여러 출력을 샘플링**해야 함 → value model 비용 대신 **생성(generation) 시간**을 소비

- **효과는 여전히 논쟁 중** (still arguable point) — 새 RL 알고리즘이 계속 제안됨:
 - **Dr. GRPO** (Liu et al., "Understanding R1-Zero-Like Training: A Critical Perspective", arXiv 2025.05): advantage에서 **표준편차로 나누는 것(division by std.)을 제거** — 이 나눗셈이 정책에 나쁜 행동을 유발함을 보임.
 - **REINFORCE++** (Hu et al., arXiv 2025.01): 출력 단위 advantage뿐 아니라 **query-wise advantage**를 활용.
- 그럼에도 구현이 쉬워 **PPO의 대안으로 널리 채택**: Llama-Nemotron(NVIDIA), Search-R1, Vision-R1(멀티모달), Robot-R1(로보틱스) 등 — reasoning 모델 학습에도 적용.

★ 시험 핵심

① GRPO 동기 = **value estimation 제거** (PPO 동기인 "급격한 갱신 방지"와 다름!). ② value가 문제인 2가지 이유(메모리·계산 / reward는 last token vs value는 every token). ③ advantage = (개별 보상 - 그룹 평균) / 그룹 std — **그룹 내부에서만**. ④ objective의 clip 구조는 PPO와 동일 + KL(ref) 정규화 포함. ⑤ value model 0개, 대신 G개 출력 샘플링 비용. ⑥ DeepSeekMath에서 제안, DeepSeek-R1에서 사용. ⑦ Dr. GRPO = std 나눗셈 제거, REINFORCE++ = query-wise advantage.

⚠ 함정 포인트 (T/F 대비)

- "GRPO trains a value model with reduced capacity to estimate advantages." → **False**. GRPO *removes* value estimation entirely; the baseline is the average reward of multiple sampled outputs in the group.
- "GRPO's motivation is the same as PPO's: preventing abrupt policy updates." → **False**. PPO's motivation is stability (limiting update size); GRPO's motivation is removing the dependency on value estimation. (GRPO는 PPO의 clip은 그대로 물려받는다.)
- "In GRPO, the advantage of an output is computed relative to rewards across the entire training dataset." → **False**. The advantage is computed using mean/std of rewards *inside each group only* (outputs sampled for the same query).

- "GRPO is computationally free compared to PPO." → **False**. It trades value-model cost for *generation cost*: multiple outputs (rollouts) must be sampled per query.
- "GRPO was first proposed in the DeepSeek-R1 paper." → **False**. It was proposed in *DeepSeekMath* (Shao et al., 2024.04); DeepSeek-R1 (2025.01) adopted it as its core RL algorithm.
- "Dr. GRPO adds a standard-deviation normalization to GRPO." → **False**. Dr. GRPO *removes* the division by the standard deviation, which was shown to induce undesirable policy behavior.
- "GRPO drops the clipping mechanism of PPO." → **False**. The clipped-ratio term is identical in structure to PPO; GRPO only changes the advantage estimation and adds KL regularization w.r.t. the reference policy.

GLM-5의 3단계 RL (GLM-5 Team, arXiv 2026.02)

RL은 LLM post-training의 핵심 패러다임. GLM-5("from Vibe Coding to Agentic Engineering")는 SFT(= RL의 **cold start**를 위한 기본 instruction tuning) 이후 **목적이 다른 3개의 RL 단계**를 순차 수행:

- 1 **Reasoning RL** — 수학(math), 과학(science), 코드(code), 기본 도구 사용(basic tool use)
- 2 **Agentic RL** — 코딩(coding) & 검색(search) 에이전트
- 3 **General RL** — 일반 RL + 정렬(alignment: RLHF/RLAIF), instruction following

→ 먼저 추론·에이전트 능력을 강화하고, **마지막 단계에서 alignment**를 수행. 핵심 RL 알고리즘은 **여전히 GRPO**(일부 문제 해결을 위한 수정 포함).

Inference / Training 분리와 vLLM

📖 교수 강조 (강의 발언)

"GRPO는 두 메커니즘이 필요하다: ① **inference**(advantage 계산용 출력 rollout), ② 계산된 advantage로 **정책 모델 갱신(training)**. inference에는 **vLLM** 같은 가속 툴킷이 많으므로, **추론과 학습을 분리하고 서로 다른 하드웨어를 쓰면** 학습을 가속할 수 있어 널리 채택된다. 그러나 이 분리(서로 다른 머신·구현)는 정책 모델과 생성 모델 사이에 **편차(deviation)**를 만들어 학습 불안정성을 유발할 수 있고, GLM-5는 이를 **distribution matching**으로 해결했다 (고급 주제)."

On-policy Distillation: 반복 RL의 망각 해결

문제: 여러 RL 단계를 반복하면 이전 단계에서 배운 지식이 **망각(forgetting)**됨 — 예: General RL을 마치면 Reasoning RL 능력이 줄어듦. **해결:** **on-policy distillation** (Agarwal et al., "On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes", ICLR 2024; Thinking Machines blog).

On-policy (cross-stage) distillation: 각 RL 단계로 튜닝된 모델을 **teacher**로 삼고 (reasoning teacher / agentic teacher / general teacher), **student 모델이 생성한 데이터**에 대해 teacher가 supervision을 제공하는 knowledge distillation. 데이터가 student에서 나오므로(on-policy) **forgetting에 더 강건**하다. 여러 teacher의 능력을 **단일 student 모델로 병합**할 수 있다.

- "특정 능력 강화용 RL 단계 도입 → 그 모델을 해당 능력의 teacher로 사용 → on-policy distillation으로 단일 모델에 병합" — 이 파이프라인은 이제 **사실상의 표준(de facto standard)**이며 GLM-4(2026.05 공개 버전)에서도 채택됨.

★ **시험 핵심**

① GLM-5의 3단계 RL 순서: Reasoning → Agentic → General(alignment는 맨 마지막). ② 핵심 알고리즘은 여전히 GRPO. ③ inference/training 분리(vLLM 가속) → 정책-생성 모델 간 deviation → 불안정성 (distribution matching으로 해결). ④ on-policy distillation: **데이터가 student에서** 나옴 → forgetting에 강건, 여러 RL teacher를 한 모델로 병합.

⚠ **함정 포인트 (T/F 대비)**

- "In GLM-5's post-training, alignment (RLHF) is performed first, before reasoning RL." → **False**. The order is Reasoning RL → Agentic RL → General RL; alignment comes in the *last* RL stage.
- "In on-policy distillation, the training data is generated by the teacher model." → **False**. The data comes from the *student* model (that is what "on-policy" means here); the teacher provides supervision on the student's generations, making it more robust to forgetting.
- "Separating inference and training machines for GRPO has no downside." → **False**. It introduces a deviation between the policy model and the generation model, which can destabilize training.
- "GLM-5 replaced GRPO with a fundamentally new RL algorithm." → **False**. The core RL algorithm is still GRPO, with modifications to resolve some issues.

★ 총정리 비교표: REINFORCE vs PPO vs GRPO

구분	REINFORCE (Vanilla PG)	PPO (-Clip)	GRPO
핵심 동기	policy gradient의 가장 기본형	급격한 갱신 방지 → 학습 안 정화 (TRPO의 간단화)	value estimation 제거 (메모 리·계산 + 토큰별 value 추정의 어려움)
Objective	$\mathbb{E}[\sum_t \nabla \log \pi_\theta \cdot \Phi_t]$ ($\Phi_t = R(\tau)$, 변 형: reward-to- go/baseline)	$\min(r_t A, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A)$	PPO와 동일한 clip 구조 + $\beta \mathbb{D}_{KL}[\pi_\theta \pi_{ref}]$ 정규화 포함
Advantage 추정	$R(\tau)$ 또는 reward-to-go (- baseline)	학습된 value model(head)로 $A = Q - V$	그룹 상대: $\hat{A}_{i,t} = \frac{r_i - \text{mean}}{\text{std}}$ (그 룹 내부에서만)
필요 모델 수 (RLHF 기 준)	policy (+ baseline 용 value 선택적)	policy + reference + value + reward (value는 보통 value head로 근사)	policy + reference + reward (value 없음)
추가 비용	높은 분산 (감소 장치 없으면)	value model 학습·추론	query당 G개 rollout 생성 시간
안정성·실무	분산 커서 불안정	안정적이나 clip만으로 보장 X → LLM은 KL penalty 병 용	PPO보다 구현 쉽고 더 안정적으 로 알려짐; 효과는 논쟁 중
대표 사용처	(교과서적 기본형)	RLHF/InstructGPT (OpenAI)	DeepSeekMath·DeepSeek- R1, GLM-5, Llama- Nemotron, Search-R1, Vision-R1, Robot-R1

변형

REINFORCE w.
baseline,
Advantage Actor-
Critic

PPO-Penalty / PPO-Clip

Dr. GRPO (std 나눗셈 제거),
REINFORCE++ (query-wise
advantage)



작년 기말 출제: Training 수 식 작성

(Problem 3 — Training of LLM, 15점)

2025 기말 **Problem 3 (Training of LLM, 15점)**은 LLM M 을 **scratch**부터 학습하는 3단계를 **손으로 수식을 직접 쓰는** 문제였다. 입력 토큰 $x = [x_1, \dots, x_L]$ (각 x_i 는 정수, $1 \leq x_i \leq |\mathcal{V}|$)에 대해 모델은 확률 $p = [p_1, \dots, p_L] \in \mathbb{R}^{L \times |\mathcal{V}|}$ 를 출력하고, $\sum_{k=1}^{|\mathcal{V}|} p_i(k) = 1$, $p_i(k) \geq 0$. 세 단계는 ① **next-token (pretraining)** → ② **instruction tuning** → ③ **RLHF** 순서.

⚠ 출제 형식 경고 (가장 큰 갭)

- 기말 Problem 3은 **객관식이 아니라 수식을 직접 손으로 쓰는 문제**(15점, 각 5점). ① **CE loss** · ② **instruction tuning loss(response 토큰만)** · ③ **RLHF objective(reward - β ·KL)** 세 수식을 외워서 쓸 수 있어야 한다.
- 함정 1 — **인덱스 범위**: instruction tuning loss는 $i = L + 1$ 부터 시작 (prompt $\tilde{x}$ 부분이 아니라 **response $\tilde{y}$ 토큰만** loss에 들어감). pretraining은 $i = 1$ 부터.
- 함정 2 — **KL 방향·대상**: RLHF의 KL penalty는 **학습 중인 $p_i(=M_\theta)$ 를 reference $p'_i(=M_{inst})$ 와 비교**하는 $KL(p_i || p'_i)$. reference는 **instruction-tuned 모델**이지 pretraining 모델이 아니다.
- 함정 3 — **부호**: pretraining/instruction은 $\min \mathbb{E}[\cdot]$ (손실 최소화, 각 항 앞 $-\log$), RLHF는 $\max_\theta \mathbb{E}[r - \beta KL]$ (보상 최대화, KL은 빼는 penalty).

① Next-token (pretraining) loss

작년 기출

문제: well-curated corpus $D_{next} = \{x^{(i)}\}_{i=1}^N$ 로 next-token 예측을 학습. 손실 함수 이름과 $\min_\theta \mathbb{E}_{x \sim D_{next}} [f]$ 형태의 수식(f 는 x, p 의 함수)을 써라. (x 의 길이는 항상 L)

정답 — (1) 이름: Cross-entropy loss · (2) 수식

$$L_{next} := \sum_{i=1}^{L-1} -\log p_i(x_{i+1})$$

위치 i 의 출력 분포 p_i 에서 **실제 다음 토큰** x_{i+1} 의 확률에 $-\log$ 를 씌운 negative log-likelihood = cross-entropy. 마지막 위치 $i = L$ 은 다음 토큰이 없으므로 합은 $i = 1$ 부터 $L - 1$ 까지.

▼ 왜 이 형태인가 (인덱스 해설)

CE = $-\sum_k y_k \log \hat{y}_k$ 에서 정답이 one-hot($y =$ 다음 토큰)이므로 $-\log p_i(x_{i+1})$ 한 항만 남는다. **모든 토큰**이 다음 토큰 예측의 정답으로 쓰이므로 $i = 1 \dots L - 1$ 전체에 loss가 걸린다 — 이것이 instruction tuning과의 결정적 차이.

② Instruction tuning loss (response 토큰만)

작년 기출

문제: M_{next} 가 instruction을 잘 못 따라서 $D_{inst} = \{(\tilde{x}^{(i)}, \tilde{y}^{(i)})\}_{i=1}^{N'}$ 로 추가 학습. $|\tilde{x}| = L, |\tilde{y}| = M$ 이고 $x := \tilde{x} \oplus \tilde{y}$ (concat). 손실 이름과 $\min_{\theta} \mathbb{E}_{(\tilde{x}, \tilde{y}) \sim D_{inst}} [g]$ 형태의 수식(g 는 x, p 의 함수)을 써라.

정답 — (1) 이름: Cross-entropy loss · (2) 수식

$$L_{inst} := \sum_{i=L+1}^{L+M-1} -\log p_i(x_{i+1})$$

CE 형태는 ①과 동일하지만 **합의 범위가 $i = L + 1$ 부터**. prompt $\tilde{x}$ (위치 $1 \dots L$) 부분은 loss에 넣지 않고 **response $\tilde{y}$ (위치 $L + 1 \dots L + M$) 토큰만** 학습한다.

▼ 왜 $i = L + 1$ 부터인가 (response 토큰만)

Instruction tuning은 모델이 **주어진 prompt에 대해 올바른 response를 생성**하도록 가르치는 것이지, prompt 자체를 재현하도록 가르치는 게 아니다. 따라서 loss mask가 prompt 토큰을 가리고 **response 부분($\tilde{y}$)만** 남긴다 → 합이 $i = L + 1$ 에서 시작. 끝은 $L + M - 1$ (마지막 토큰 $L + M$ 은 다음 토큰이 없으므로). 시험에서 $i = 1$ 부터 쓰면 **오답** — 이 인덱스가 채점 포인트.

③ RLHF objective (reward – β ·KL)

작년 기출

문제: M_{inst} 가 위험·유해 응답을 내서 RLHF로 추가 fine-tune. $D_{RLHF} = \{\hat{x}^{(i)}\}_{i=1}^{N''}$ 와 reward model $r_{\phi} : x \rightarrow [0, 1]$ ($x := \hat{x} \oplus \hat{y}, |\hat{x}| = L, |\hat{y}| = M$). p' 는 M_{inst} 의 출력 확률. $\max_{\theta} \mathbb{E}_{\hat{x} \sim D_{RLHF}, \hat{y} \sim M_{\theta}(\hat{x})} [h - \beta h']$ 형태로 써라 (h 는 x, r_{ϕ} 의 함수, h' 는 x, p, p' 의 함수).

정답 — (1) $h = r_{\phi}(x)$ · (2) $h' = \text{KL penalty}$

$$\max_{\theta} \mathbb{E}_{\hat{x} \sim D_{RLHF}, \hat{y} \sim M_{\theta}(\hat{x})} \left[r_{\phi}(x) - \beta \sum_{i=L+1}^{L+M-1} \text{KL}(p_i \| p'_i) \right]$$

$$h = r_\phi(x) \text{ (응답 품질 보상, 한 번 부여)} - \beta \cdot h' \text{ (정규화 penalty). KL 항을 풀어 쓰면 } h' = \sum_{i=L+1}^{L+M-1} \sum_k p_i(k) \log \frac{p_i(k)}{p'_i(k)} = \sum_{i=L+1}^{L+M-1} \text{KL}(p_i \| p'_i).$$

▼ 왜 max·왜 KL이 reference M_{inst} 와인가

- **max·부호**: RLHF는 reward를 **최대화**하는 objective (pretraining/instruction의 **min** loss와 반대). $-\beta h'$ 는 reward 해킹을 막는 penalty이므로 빼준다.
- **KL 대상 = $M_{inst}(=p')$** : reference policy는 **방금 instruction tuning을 마친 모델**. RLHF로 갱신되는 $M_\theta(=p)$ 가 reference에서 너무 멀어지면 언어 능력을 망각(forgetting)하므로 $\text{KL}(p \| p')$ 로 묶어둔다. pretraining 모델이 아니라 M_{inst} 가 reference인 것이 함정.
- **KL 방향**: $\text{KL}(p_i \| p'_i)$ — 학습 중 분포 p 가 앞, reference p' 가 뒤. (본 장 본문 PPO/GRPO의 $\mathbb{D}_{KL}[\pi_\theta \| \pi_{ref}]$ 와 동일한 방향.)
- **범위 $i = L + 1 \dots L + M - 1$** : ②와 동일하게 **response 토큰**에 대해서만 KL을 건다 (prompt는 고정 입력).

✂ 한 장 암기 — 세 수식 나란히

- ① Pretraining (CE): $L_{next} = \sum_{i=1}^{L-1} -\log p_i(x_{i+1})$ — **모든 토큰**, min.
- ② Instruction (CE, response만): $L_{inst} = \sum_{i=L+1}^{L+M-1} -\log p_i(x_{i+1})$ — $i = L + 1$ 부터, min.
- ③ RLHF: $\max_\theta \mathbb{E} \left[r_\phi(x) - \beta \sum_{i=L+1}^{L+M-1} \text{KL}(p_i \| p'_i) \right]$ — reward — $\beta \cdot \text{KL}$ (vs M_{inst}), max.



예상문제 (직접 기출 없음 — 중간고사 이후 범위)

2025 중간 기출 형식 그대로: T/F (정답 3점·오답 0점·**무응답 1점**) + "Pick all that are correct; any wrong answer will lead to 0 points".

Part A-0. 작년 기말 Problem 1 직접 기출 (RL 관련 T/F)

P1-(i). (True/False) 작년 기출

RLHF provides an interpretable rule set defining what "good behavior" is.

▼ 정답 & 해설 보기

False. RLHF defines "goodness" **implicitly** via large human preference datasets, **not** an interpretable rule set. (규칙을 명시하는 게 아니라 사람 선호 데이터로 암묵적으로 정의.)

P1-(ix). (True/False) 작년 기출

The motivation of PPO is to take biggest possible improvement step on policy, without stepping so far from current policy to prevent collapse.

▼ 정답 & 해설 보기

True. PPO의 동기 그대로 — "현재 정책에서 너무 멀리 벗어나 붕괴(collapse)하지 않으면서 가능한 가장 큰 개선 step". (단, clip만으로는 보장되지 않음 → LLM에선 KL penalty 병용.)

P1-(x). (True/False) 작년 기출

GRPO estimates advantage function by generating multiple responses per query and calculating the statistics of their rewards.

▼ 정답 & 해설 보기

True. GRPO는 query당 여러 응답(group)을 생성하고 그 보상들의 통계(그룹 평균·표준편차)로 advantage $\hat{A}_{i,t} = \frac{r_i - \text{mean}}{\text{std}}$ 를 추정 — value model 없이.

Part A. True/False (예상문제)

Q1. (True/False) 예상문제

When an LLM is trained with reinforcement learning, the LLM corresponds to the policy, the input tokens correspond to the state, and generating the next token corresponds to the action.

▼ 정답 & 해설 보기

True. This is exactly the mapping given in the lecture: (1) policy = LLM, (2) state = input tokens, (3) action = generation of the next token.

Q2. (True/False) 예상문제

Adding a state-dependent baseline $b(s_t)$ to the policy gradient reduces the bias of the gradient estimate.

▼ 정답 & 해설 보기

False. A baseline does not change the bias at all: since $\mathbb{E}_{a_t \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t)] = 0$, the expected gradient is unchanged. What a good baseline (most commonly the on-policy value function $V^\pi(s_t)$) reduces is the *variance* of the estimate.

Q3. (True/False) 예상문제

In the PPO-Clip objective, when the advantage is positive, taking the min with the clipped ratio limits how much the objective can increase, so the new policy does not benefit from moving far away from the old policy.

▼ 정답 & 해설 보기

True. With $A > 0$, the objective becomes $\min(\pi_\theta/\pi_{\theta_k}, 1 + \epsilon) A$; the min caps the increase, removing any benefit from pushing the ratio beyond $1 + \epsilon$. (Symmetrically, with $A < 0$ the max with $1 - \epsilon$ limits the decrease.)

Q4. (True/False) 예상문제

GRPO estimates the advantage of each sampled output using a separate learned value model, normalized by the group standard deviation.

▼ 정답 & 해설 보기

False. GRPO removes the value model entirely. The advantage is computed purely from the rewards within the group: $\hat{A}_{i,t} = (r_i - \text{mean}\{r_j\})/\text{std}\{r_j\}$, where the group is the set of outputs sampled from the old policy for the same query. No value model is involved.

Q5. (True/False) 예상문제

One motivation for GRPO is that in LLMs the reward is usually computed from the last token, whereas the value must be estimated for each token, which makes accurate value estimation complicated.

▼ 정답 & 해설 보기

True. This is the second reason given in the slides (the first being the substantial memory/computational burden of a value model). A single token's generation may look good now but turn out bad several steps later, so per-token value estimation is unintuitive in text.

Q6. (True/False) 예상문제

The reward-to-go policy gradient at time step t sums the rewards over the entire trajectory, including rewards received before the action a_t was taken.

▼ 정답 & 해설 보기

False. Reward-to-go sums rewards only from step t onward ($\sum_{t'=t}^T r_{t'}$). Rewards obtained before the action cannot be affected by it, so including them (as the basic policy gradient does with $R(\tau)$) is less accurate.

Q7. (True/False) 예상문제

PPO was developed as a simpler alternative to TRPO, which answers the same question (taking the biggest possible improvement step without moving too far from the current policy) but is much more complex and computationally expensive.

▼ 정답 & 해설 보기

True. TRPO (Schulman et al., ICML 2015) addresses the same stability question but is 3–4× more expensive and hard to apply. John Schulman (OpenAI) developed PPO with two variants: PPO-Penalty and PPO-Clip.

Q8. (True/False) 예상문제

In GLM-5's post-training pipeline, on-policy distillation uses data generated by the teacher models so that the student can imitate the teachers' outputs directly.

▼ 정답 & 해설 보기

False. In *on-policy* distillation, the data comes from the *student* model's own generations, and the RL-tuned teachers provide supervision on those generations. This is precisely why it is more robust to the forgetting issue, and it allows merging multiple teachers' capabilities into a single student model.

Q9. (True/False) 예상문제

Because the gradients of the environment functions (the initial state distribution and the transition probabilities) with respect to θ are zero, the gradient of the log-probability of a trajectory reduces to the sum of the gradients of the log policy probabilities.

▼ 정답 & 해설 보기

True. The parameter θ appears only in the policy, so $\nabla_{\theta} \log \rho_0(s_0) = 0$ and $\nabla_{\theta} \log P(s_{t+1}|s_t, a_t) = 0$, leaving $\nabla_{\theta} \log P(\tau|\theta) = \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$.

Part B. Pick all that are correct (any wrong answer → 0 points)

Q10. (Pick all that are correct) 예상문제

Which of the following are true about the key concepts of reinforcement learning?

- a. The return is the cumulative reward over a trajectory, and the agent's goal is to maximize it.
- b. The value function $V^{\pi}(s)$ is conditioned on both a specific state and a specific action.
- c. Taking the expectation of the Q function over actions sampled from the policy yields the value function.
- d. For an infinite-horizon return, a discount factor $\gamma \in (0, 1)$ is introduced to keep the sum from diverging.

▼ 정답 & 해설 보기

정답: (a), (c), (d)

- (a) Correct. Return = cumulative reward; maximizing it is the agent's goal.
- (b) Incorrect. The *value* function is conditioned on a state only; the Q function is conditioned on a state and an action.
- (c) Correct. $V^{\pi}(s) = \mathbb{E}_{a \sim \pi}[Q^{\pi}(s, a)]$ — the natural relationship.
- (d) Correct. Without γ , the infinite sum may diverge to infinity.

Q11. (Pick all that are correct) 예상문제

Which of the following are true about the derivation of the basic policy gradient?

- a. The log-derivative trick rewrites $\nabla_{\theta} P(\tau|\theta)$ as $P(\tau|\theta) \nabla_{\theta} \log P(\tau|\theta)$.

- b. The resulting policy gradient is $\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} [\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau)]$.
- c. The expectation is computed exactly in closed form during training.
- d. The sample estimate uses trajectories collected from the current policy π_{θ} .

▼ 정답 & 해설 보기

정답: (a), (b), (d)

- (a) Correct. It follows from $\nabla \log f = \nabla f / f$.
- (b) Correct. This is the basic policy gradient formula.
- (c) Incorrect. The expectation cannot be computed in closed form for complex distributions; it is replaced with an empirical sample mean (Monte Carlo estimation).
- (d) Correct. $\mathcal{D} = \{\tau_i\}$ with $\tau_i \sim \pi_{\theta}$ — the trajectories must come from the current policy (on-policy).

Q12. (Pick all that are correct) 예상문제

Which of the following are true about the PPO-Clip algorithm?

- a. The objective takes the min between $r_t(\theta)A$ and $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A$, where $r_t(\theta)$ is the probability ratio between the current and the old policy.
- b. When the advantage is negative, the max with $(1 - \epsilon)$ limits how much the objective can decrease.
- c. PPO no longer needs a value model because clipping replaces advantage estimation.
- d. Even with clipping, the new policy can still end up too far from the old policy, so additional tricks are widely adopted in practice.

▼ 정답 & 해설 보기

정답: (a), (b), (d)

- (a) Correct. This is the PPO-Clip objective with $r_t(\theta) = \pi_{\theta}(a|s) / \pi_{\theta_k}(a|s)$.
- (b) Correct. For $A < 0$ the objective becomes $\max(r_t, 1 - \epsilon) A$, bounding the decrease.

- (c) Incorrect. Clipping constrains the update size; it does not estimate advantages. PPO still requires a learned value function to compute $A = Q - V$, which is its main bottleneck.
- (d) Correct. The slides explicitly note this limitation.

Q13. (Pick all that are correct) 예상문제

Which of the following are true about PPO as implemented for RLHF in the TRL library?

- a. TRL is a full-stack library released by Hugging Face, and algorithms can be switched by changing only the config and trainer functions.
- b. The rollout step that generates multiple responses per query is executed under `torch.no_grad`, so it produces no gradients.
- c. To avoid loading a fully separate value model, a value head is added on top of the LLM, analogous to the LM head.
- d. The LM head converts hidden representations into scalar value estimates.

▼ 정답 & 해설 보기

정답: (a), (b), (c)

- (a) Correct. E.g., switching to DPO only requires `DPOConfig` + `DPOTrainer`.
- (b) Correct. Rollout is pure sampling; no model parameters are updated there.
- (c) Correct. Loading policy + reference + value + reward as four separate models is too expensive, so the value model is approximated by a value head.
- (d) Incorrect. The LM head maps to a probability distribution over the *vocabulary*; it is the *value head* that maps to values.

Q14. (Pick all that are correct) 예상문제

Which of the following are true when comparing PPO and GRPO?

- a. Both PPO and GRPO use a clipped probability-ratio objective.
- b. GRPO replaces the value function's role with the average reward of a group of sampled outputs.

- c. GRPO eliminates all additional computational cost relative to PPO.
- d. GRPO's objective additionally includes a KL regularization term with respect to the reference policy.

▼ 정답 & 해설 보기

정답: (a), (b), (d)

- (a) Correct. The inner min/clip term of GRPO is structurally identical to PPO-Clip.
- (b) Correct. The essence of the advantage is estimating the *relative* quality of the current generation, so the group mean can serve as the baseline.
- (c) Incorrect. GRPO trades value-model cost for generation cost: multiple outputs must be sampled per query to update on a single query.
- (d) Correct. $-\beta \mathbb{D}_{KL}[\pi_{\theta} || \pi_{ref}]$ is part of the GRPO objective (with a different KL estimator than the vanilla one).

Q15. (Pick all that are correct) 예상문제

Which of the following are true about recent variants and adoption of GRPO?

- a. Dr. GRPO removes the division by the standard deviation in the advantage computation.
- b. REINFORCE++ utilizes a query-wise advantage in addition to the output-wise advantage.
- c. The effectiveness of GRPO is firmly established and no longer debated.
- d. GRPO has been adopted as an alternative to PPO in models such as Llama-Nemotron, Search-R1, Vision-R1, and Robot-R1.

▼ 정답 & 해설 보기

정답: (a), (b), (d)

- (a) Correct. The std division was shown to induce undesirable policy behavior.
- (b) Correct. That is REINFORCE++'s modification.

- (c) Incorrect. The slides state the effectiveness of GRPO is "still an arguable point," and new RL algorithms are continuously proposed. It is widely adopted mainly because of its easy implementation.
- (d) Correct. All four are listed in the slides as adopters.

Q16. (Pick all that are correct) 예상문제

Which of the following are true about LLM post-training in 2026 (GLM-5)?

- a. GLM-5 performs three RL stages in the order: Reasoning RL → Agentic RL → General RL.
- b. The General RL stage covers generic RL and alignment (RLHF/RLAIF).
- c. GLM-5 abandoned GRPO in favor of vanilla PPO.
- d. Separating the inference (rollout) machine from the training machine can introduce a deviation between the policy model and the generation model, causing training instability.

▼ 정답 & 해설 보기

정답: (a), (b), (d)

- (a) Correct. Reasoning (math/science/code/basic tool use) → Agentic (coding & search) → General.
- (b) Correct. Alignment comes last, after reasoning and agentic capabilities are strengthened.
- (c) Incorrect. The core RL algorithm in GLM-5 is still GRPO (with modifications to resolve some issues).
- (d) Correct. Using different machines/implementations (e.g., vLLM for inference) speeds up training but introduces a policy–generation deviation, addressed by distribution matching.

Q17. (Pick all that are correct) 예상문제

Which of the following are true about variance reduction in policy gradient methods?

- a. For any function of the state $b(s_t)$, $\mathbb{E}_{a_t \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t)] = 0$.
- b. The most common choice of baseline is the on-policy value function $V^\pi(s_t)$.
- c. The advantage function is defined as $A^\pi(s_t, a_t) = V^\pi(s_t) - Q^\pi(s_t, a_t)$.
- d. Depending on the choice of Φ_t , the algorithm is called REINFORCE (cumulative reward), REINFORCE with baseline, or an advantage actor-critic algorithm.

▼ 정답 & 해설 보기

정답: (a), (b), (d)

- (a) Correct. This identity is why inserting a baseline leaves the expected gradient unchanged.
- (b) Correct. Empirically known to be effective for reducing the variance of the estimate.
- (c) Incorrect. The advantage is $A^\pi = Q^\pi - V^\pi$ — how much better the action is than the average — not $V - Q$.
- (d) Correct. The name of the algorithm varies with the choice of Φ_t .

3분 요약

개념	한 줄 정리
RL 기본 매핑	policy=LLM, state=input tokens, action=next-token generation; 목표는 return(누적 보상) 최대화
J(θ) & Policy Gradient	$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$ 를 gradient ascent 로 최대화; PPO·GRPO 모두 policy gradient 계열
기본 PG 결과식	$\nabla J = \mathbb{E}[\sum_t \nabla \log \pi_\theta(a_t s_t) R(\tau)]$ — log-derivative trick + 환경항 gradient=0 + Monte Carlo 추정
분산 감소 3종	reward-to-go(t 이후만) / baseline(기댓값 불변·분산↓, 보통 V^π) / advantage $A = Q - V$ (평균 대비 우수성)
PPO-Clip	$\min(rA, \text{clip}(r, 1-\epsilon, 1+\epsilon)A)$ — $A>0$ 이면 min이 상한, $A<0$ 이면 max가 하한; value model 필요; LLM에선 KL penalty 병용
TRL 구현	HF의 TRL(빠른 대안 veRL); rollout은 torch.no_grad; 별도 value model 대신 value head (LM head=vocab 확률, value head=가치)
GRPO	value model 제거, 그룹 G개 rollout의 $\hat{A} = (r_i - \text{mean})/\text{std}$; clip 구조는 PPO와 동일 + KL(ref); 대신 생성 비용 증가; DeepSeekMath 제안→R1 사용
GRPO 변형	Dr. GRPO=std 나눗셈 제거, REINFORCE++=query-wise advantage; 효과는 논쟁 중이나 구현 쉬워 널리 채택
2026 Post-training	GLM-5: SFT(cold start)→Reasoning RL→Agentic RL→General RL(alignment 마지막), 핵심은 여전히 GRPO; vLLM 추론/학습 분리→deviation 문제
On-policy Distillation	RL 단계별 모델을 teacher로, student가 생성한 데이터 에 supervision → forgetting에 강건, 능력 병합 (de facto standard)

[← 전체 목차](#)

[다음 장 →](#)